GSFC UNIVERSITY
EDUCATION RE-ENVISIONED

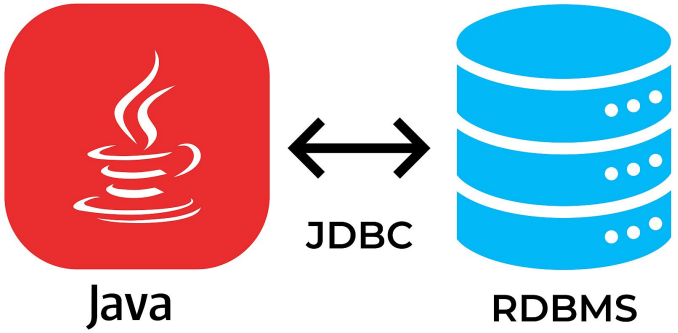
JDBC Programming

Ms.Shweta Rajput

Department of Computer Science & Engineering

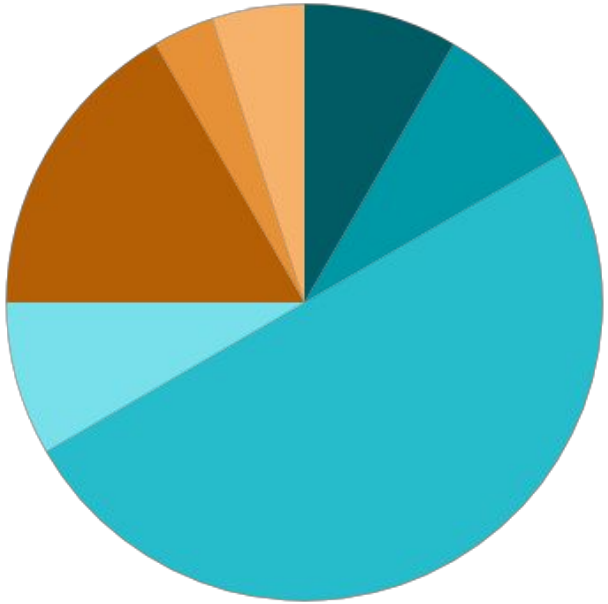
Outline

1. Introduction & Background
2. JDBC Connectivity Model, Connecting to Database, Creating & Executing SQL Queries
3. JDBC Interfaces: Statement, PreparedStatement, CallableStatement
4. Transaction Management
5. Retrieving & Updating Data, Error Handling: SQLException, SQLWarning



Source of Image: https://darvishdarab.github.io/cs421_f20/docs/readings/db/jdbc/

Lecture Structure



05 Minutes	Outline & Objectives
05 Minutes	Recap of Previous Lecture
30 Minutes	Delivery of Actual Content as per Lesson Plan
05 Minutes	Industry Relevance
10 Minutes	Interactive Activities
02 Minutes	Importance of Discipline
03 Minutes	Recording of Attendance

Reference: Office Order No. Provost/092023135, Date: 18th September 2023

Objectives

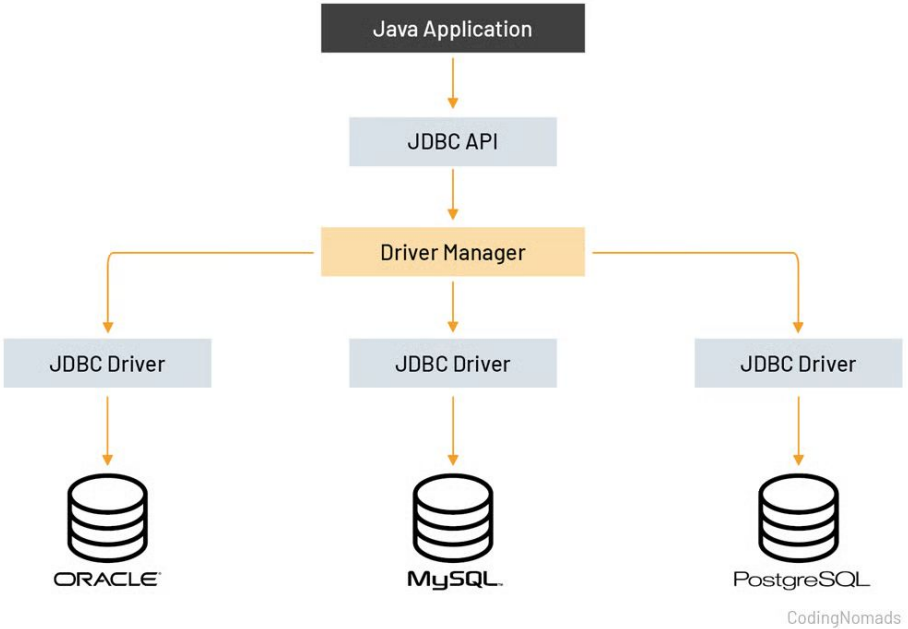
1. Understand the basics and need of JDBC
2. Learn the JDBC connectivity model and database connection process
3. Create and execute SQL queries using Java
4. Use JDBC interfaces: Statement, PreparedStatement, and CallableStatement
5. Implement transaction management for reliable database operations

Understand the basics

- Enterprise applications that are created using the Java EE technology need to interact with databases to store — application-specific information.
- Interacting with database requires database connectivity, which can be achieved by using the Open Database Connectivity (ODBC) driver. This driver is used with Java Database Connectivity (JDBC) to interact with various types of databases, such as Oracle, MS Access, MySQL, and sQL Server.
- JDBC is an Application Programming Interface (API), which is used in Java programming to interact with databases.
- JDBC works with different database drivers to connect to different databases.

JDBC - Java Database Connectivity

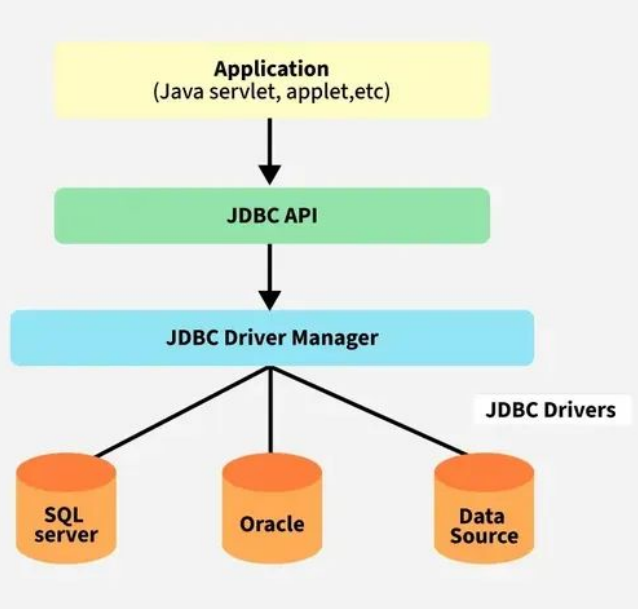
1. Understand the basics and need of JDBC
2.



https://codingnomads.com/java-301-what-is-jdbc

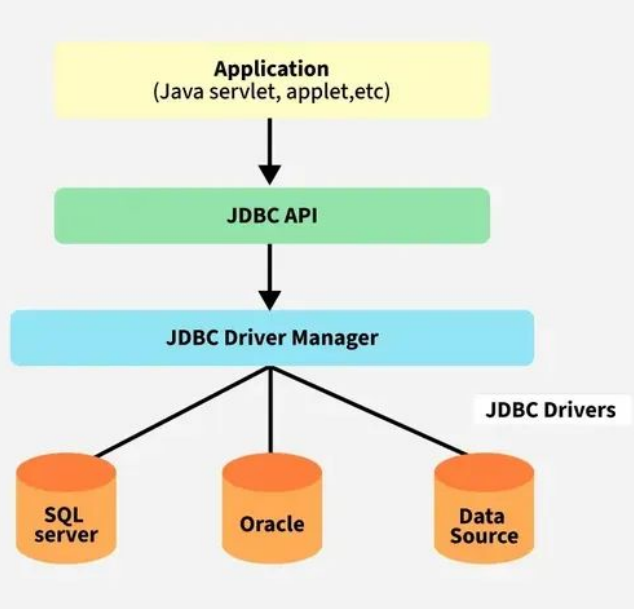
JDBC Architecture

- Application: It can be a Java application or servlet that communicates with a data source.
- JDBC API: It allows Java programs to execute SQL queries and get results from the database.
  - Some key components of JDBC API include Interfaces like Driver, ResultSet, RowSet, PreparedStatement and Connection that helps managing different database tasks.
  - Classes like DriverManager, Types, Blob and Clob that helps managing database connections.



JDBC Architecture

- DriverManager: It plays an important role in the JDBC architecture. It uses some database-specific drivers to effectively connect enterprise applications to databases.
- JDBC drivers: These drivers handle interactions between the application and the database.

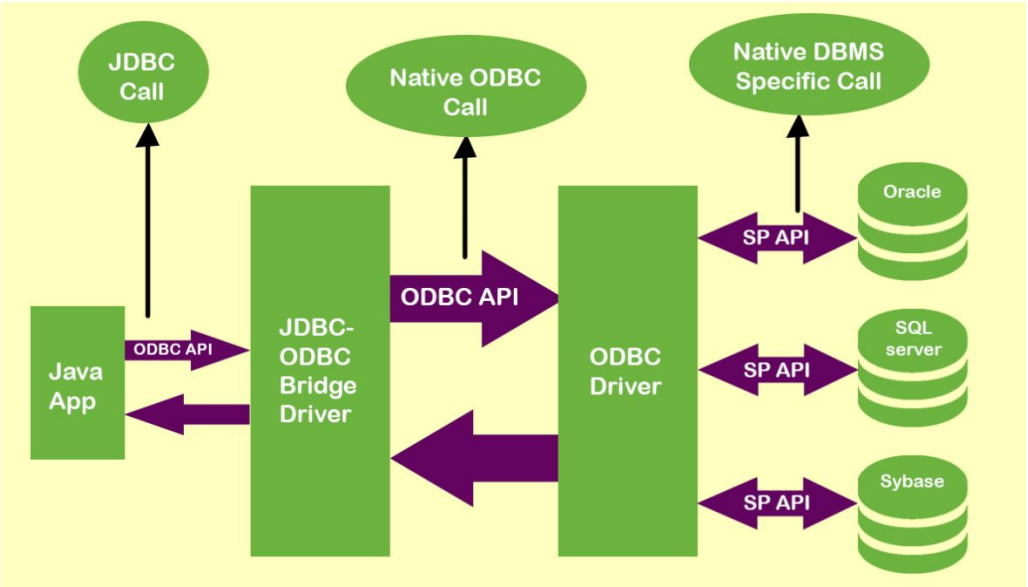


JDBC Driver

- Type - 1
- Type - 2
- Type - 3
- Type - 4

JDBC Type -1 Driver

- The Type-1 driver acts as a bridge between JDBC and other database connectivity mechanisms, like ODBC.
- Example: Sun JDBC-ODBC bridge driver, which provides access to the database through the ODBC drivers.



https://erainnovator.com/jdbc-drivers/

JDBC Type -1 Driver

Steps that are involved in establishing a connection between a Java application and data source

a. The Java application makes the JDBC call to the JDBC-ODBC bridge driver to access a data source.
b. The JDBC-ODBC bridge driver resolves the JDBC call and makes an equivalent ODBC call to the ODBC driver.
c. The ODBC driver completes the request and sends responses to the JDBC-ODBC bridge driver.
d. The JDBC-ODBC bridge driver converts the response into JDBC standards and displays the result to the requesting Java application.

JDBC Type -1 Driver

Advantages:

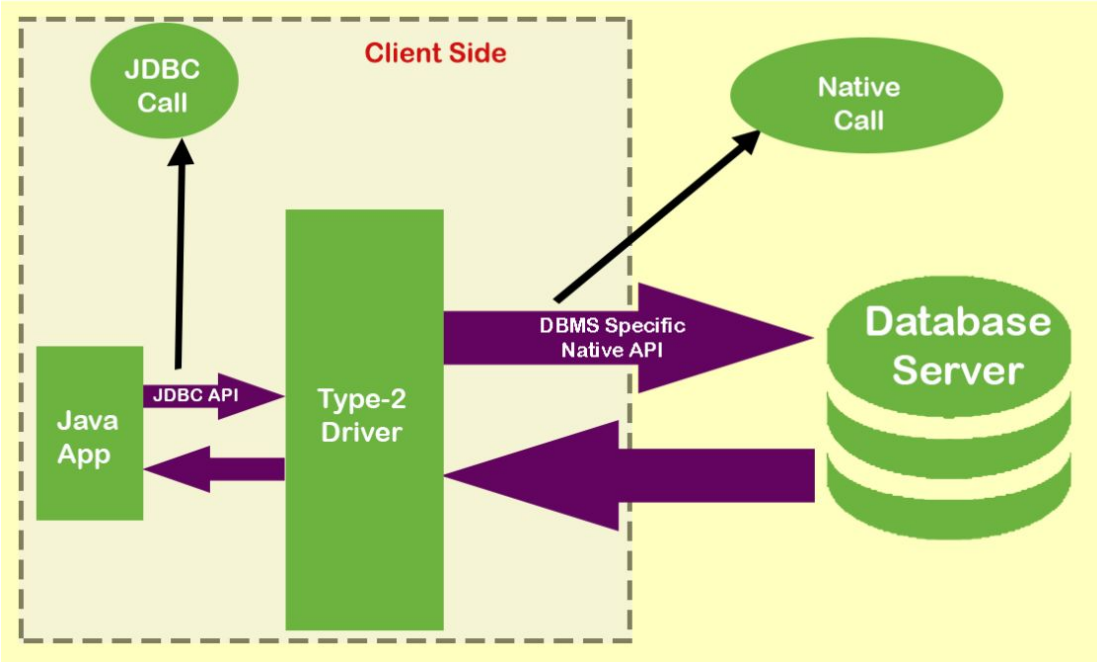
- Represents single driver implementation to interact with different data sources
- Allows you to communicate with all the databases supported by the ODBC driver
- Represents a vendor independent driver

Disadvantages:

- Decreases the execution speed due to a large number of translations
- Depends on the ODBC driver, and thus, Java applications also become indirectly hooked into ODBC drivers
- Requires the ODBC binary code or ODBC client library that must be installed on every client Uses Java Native Interface (JNI) to make ODBC calls

JDBC Type -2 Driver

- Type-2 Driver (Java to Native API) The JDBC call can be converted into the database vendor-specific native call with the help of the Type-2 driver.
- Driver makes JNI calls on database-specific native client API.
- Database-specific native client APIs are usually written in C and C++.



https://erainnovator.com/jdbc-drivers/

JDBC Type -2 Driver

Advantages:

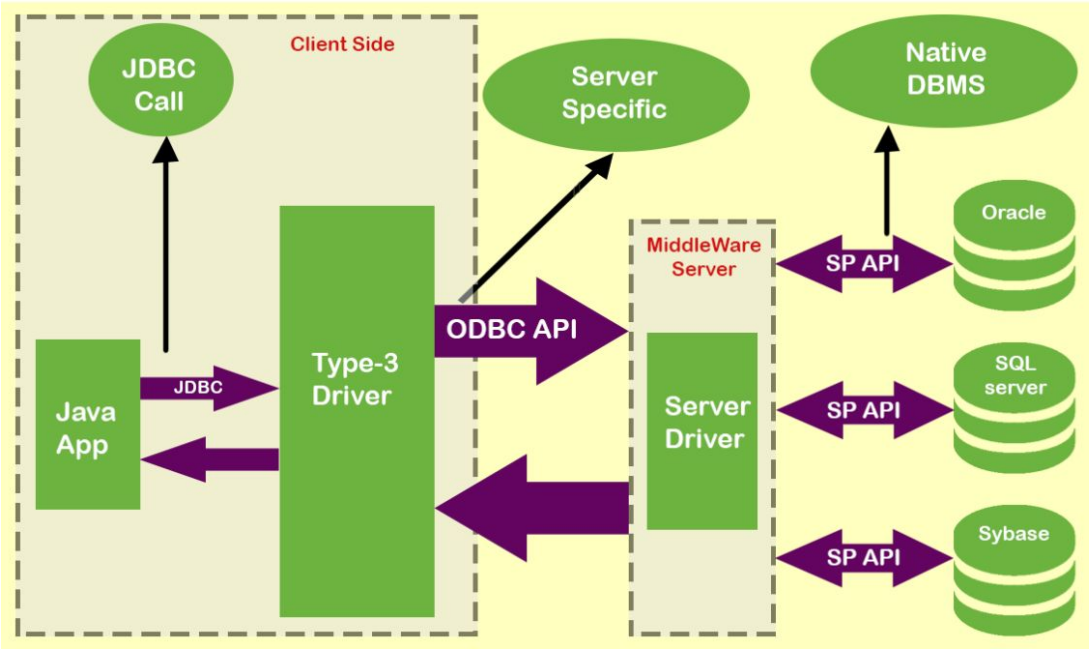
- Helps to access the data faster as compared to other types of drivers
- Contains additional features provided by the specific database vendor, which are also supported by the JDBC specification

Disadvantages:

- Requires native libraries to be installed on client machines since the conversion from JDBC calls to database-specific native calls is done on client machines
- Executes the database-specific native functions on the client JVM, implying that any bug in the Type-2 driver might crash the JVM
- Increases the cost of the application in case it is run on different platforms

JDBC Type -3 Driver

- Also known as net protocol drivers
- The Type-3 driver translates the JDBC calls into a database server independent and middleware server-specific calls.
- The translated JDBC calls are further translated into database server specific calls.



https://erainnovator.com/jdbc-drivers/

JDBC Type -3 Driver

Advantages:

- Serves as a Java driver and is auto downloadable.
- Does not require any native library to be installed on the client machine.
- Database independence - as a single driver provides access to different types of databases.
- Does not provide the database details, such as username, password, and database server location, to the client. These details are automatically configured in the middleware server.
- Provides the facility to switch over from one database to another without changing the client-side driver classes. Switching of databases can be implemented by changing the configurations of the middleware server.

JDBC Type -3 Driver

Disadvantages:

- It performs the tasks slowly due to the increased number of network calls as compared to Type-2 drivers.
- In addition, the Type-3 driver is also costlier as compared to other drivers.

Examples:

- IDS Driver
- Weblogic RMI Driver

JDBC Type -4 Driver

- Known as Java to Database Protocol
- A pure Java driver, which implements the database protocol to interact directly with a database.
- Does not require any native database library to retrieve the records from the database.
- Type-4 driver translates JDBC calls into database specific network calls.

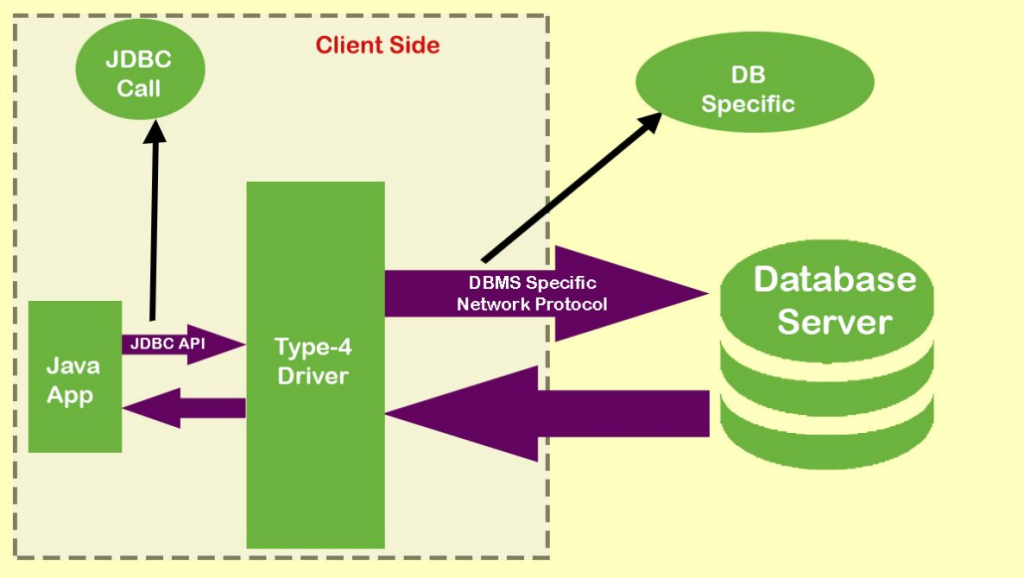


Image source : https://erainnovator.com/jdbc-drivers/

JDBC Type -4 Driver

- Known as Java to Database Protocol
- A pure Java driver, which implements the database protocol to interact directly with a database.
- Does not require any native database library to retrieve the records from the database.
- Type-4 driver translates JDBC calls into database specific network calls.

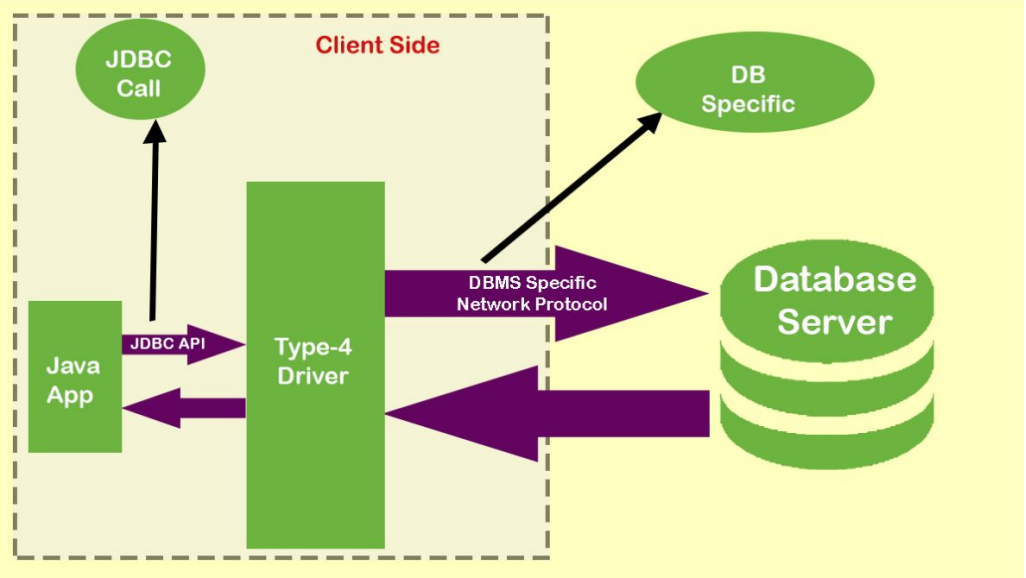


Image source : https://erainnovator.com/jdbc-drivers/

JDBC Type -4 Driver

Advantages:

- Serves as a pure Java driver and is auto downloadable
- Does not require any native library to be installed on the client machine
- Uses database server specific protocol
- Does not require a middleware server

Disadvantages:

- The main disadvantage of the Type-4 driver is that it uses database specific proprietary protocol and is DBMS vendor dependent.

Examples:

- Thin Driver for Oracle from Oracle Corporation
- Weblogic and MSSQLServer4 for MS SQL server from BEA systems.

JDBC Type -4 Driver

Advantages:

- Serves as a pure Java driver and is auto downloadable
- Does not require any native library to be installed on the client machine
- Uses database server specific protocol
- Does not require a middleware server

Disadvantages:

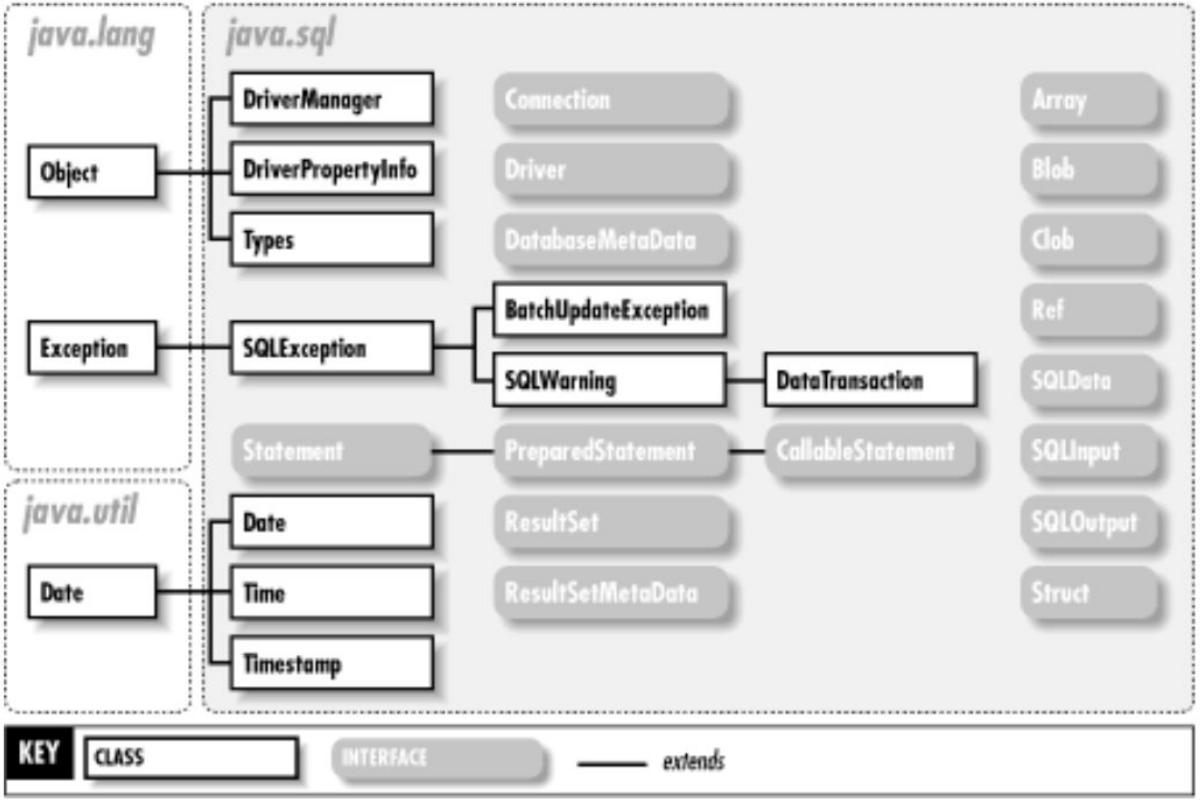
- The main disadvantage of the Type-4 driver is that it uses database specific proprietary protocol and is DBMS vendor dependent.

Examples:

- Thin Driver for Oracle from Oracle Corporation
- Weblogic and MSSQLServer4 for MS SQL server from BEA systems.

java.sql package

Contains the entire JDBC API that sends SQL (Structured Query Language) statements to relational databases and retrieves the results of executing those SQL statements



DriverManager Class

- The basic service for managing a set of JDBC drivers.
- DriverManager class will attempt to load the driver classes referenced in the "jdbc.drivers" system property.

Modifier and Type	Method	Description
`static Connection`	`getConnection(String url, String user, String password)`	Attempts to establish a connection to the given database URL.
`static Driver`	`getDriver(String url)`	Attempts to locate a driver that understands the given URL

Driver

- The interface that every driver class must implement.
- The Java SQL framework allows for multiple database drivers.
- Each driver should supply a class that implements the Driver interface.
- The DriverManager will try to load as many drivers as it can find and then for any given connection request, it will ask each driver in turn to try to connect to the target URL.

Type	Method	Description
`int`	**`getMajorVersion`**`()`	Retrieves the driver's major version number.
`int`	**`getMinorVersion`**`()`	Gets the driver's minor version number.

Connection Interface

- A connection (session) with a specific database.
- SQL statements are executed and results are returned within the context of a connection.
- A Connection object's database is able to provide information describing its tables, its supported SQL grammar, its stored procedures, the capabilities of this connection, and so on.

Connection Interface

Modifier and Type	Method	Description
`void`	**`close`**`()`	Releases this `Connection` object's database and JDBC resources immediately instead of waiting for them to be automatically released.
`Statement`	**`createStatement`**`()`	Creates a `Statement` object for sending SQL statements to the database.
`boolean`	**`isClosed`**`()`	Retrieves whether this `Connection` object has been closed.
`CallableStatement`	**`prepareCall`**`(`**`String`** `sql)`	Creates a `CallableStatement` object for calling database stored procedures.
`PreparedStatement`	**`prepareStatement`**`(`**`String`** `sql)`	Creates a `PreparedStatement` object for sending parameterized SQL statements to the database.

Statement Interface

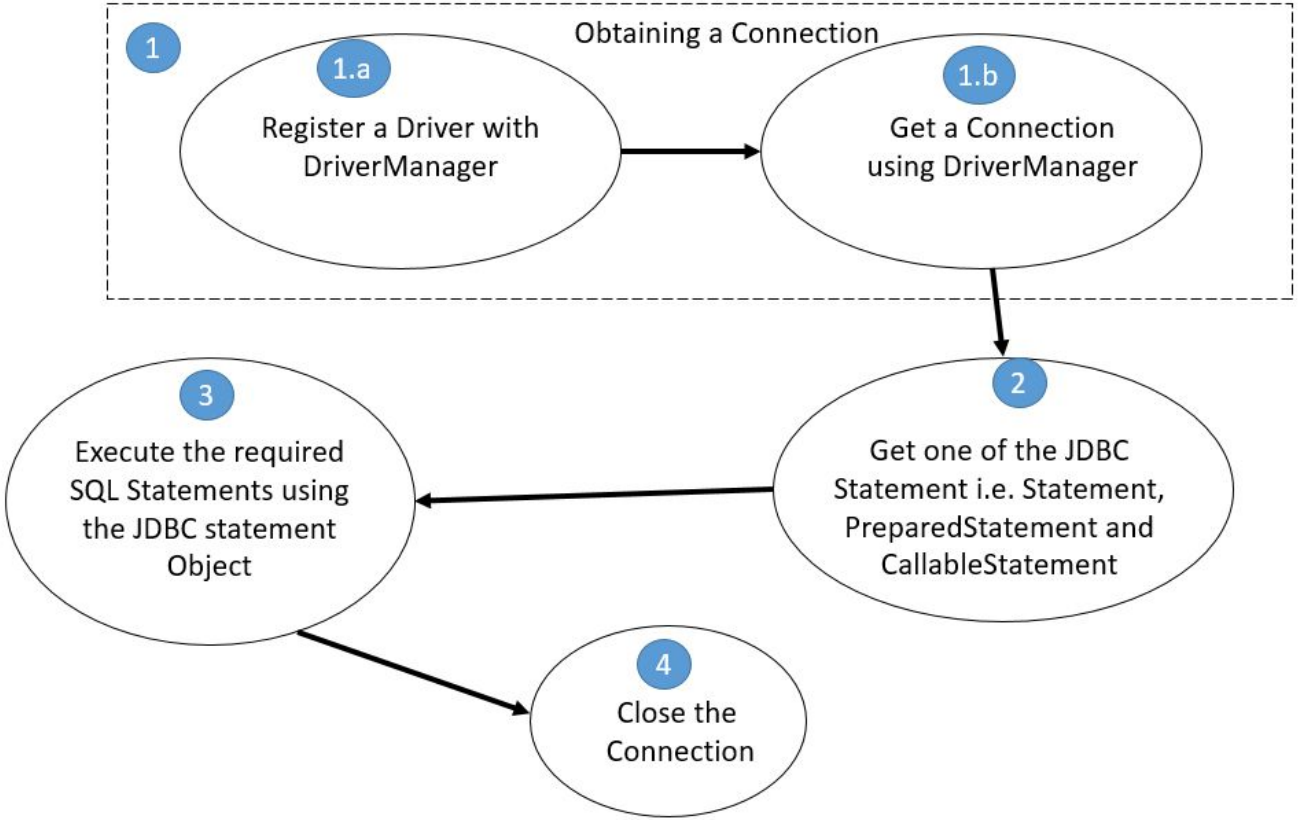
- The object used for executing a static SQL statement and returning the results it produces.

Type	Method	Description
`boolean`	**`execute`**`(`**`String`**` sql)`	Executes the given SQL statement, which may return multiple results.
`ResultSet`	**`executeQuery`**`(`**`String`**` sql)`	Executes the given SQL statement, which returns a single `ResultSet` object.
`int`	**`executeUpdate`**`(`**`String`**` sql)`	Executes the given SQL statement, which may be an `INSERT`, `UPDATE`, or `DELETE` statement or an SQL statement that returns nothing, such as an SQL DDL statement.
`void`	**`close`**`()`	Releases this `Statement` object's database and JDBC resources immediately instead of waiting for this to happen when it is automatically closed.

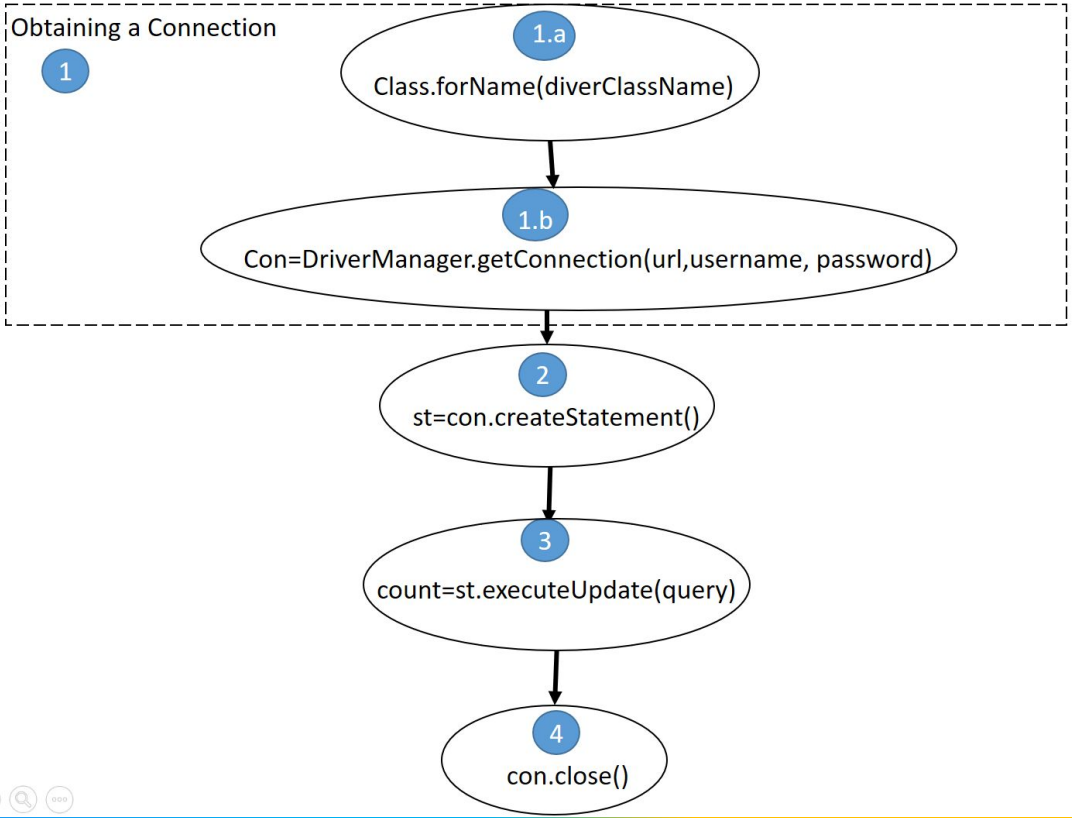
Understanding Basic JDBC Steps

- To establish a connection with a database and retrieve the desired results, you need to perform various steps. For example, you need to register a driver with the DriverManager object, obtain a connection, and execute SQL queries.
- The following broad steps that need to be performed to implement JDBC in Java application:
  1. Obtaining a connection
  2. Creating a JDBC Statement object
  3. Executing SQL statements
  4. Closing the connection

Understanding Basic JDBC Steps



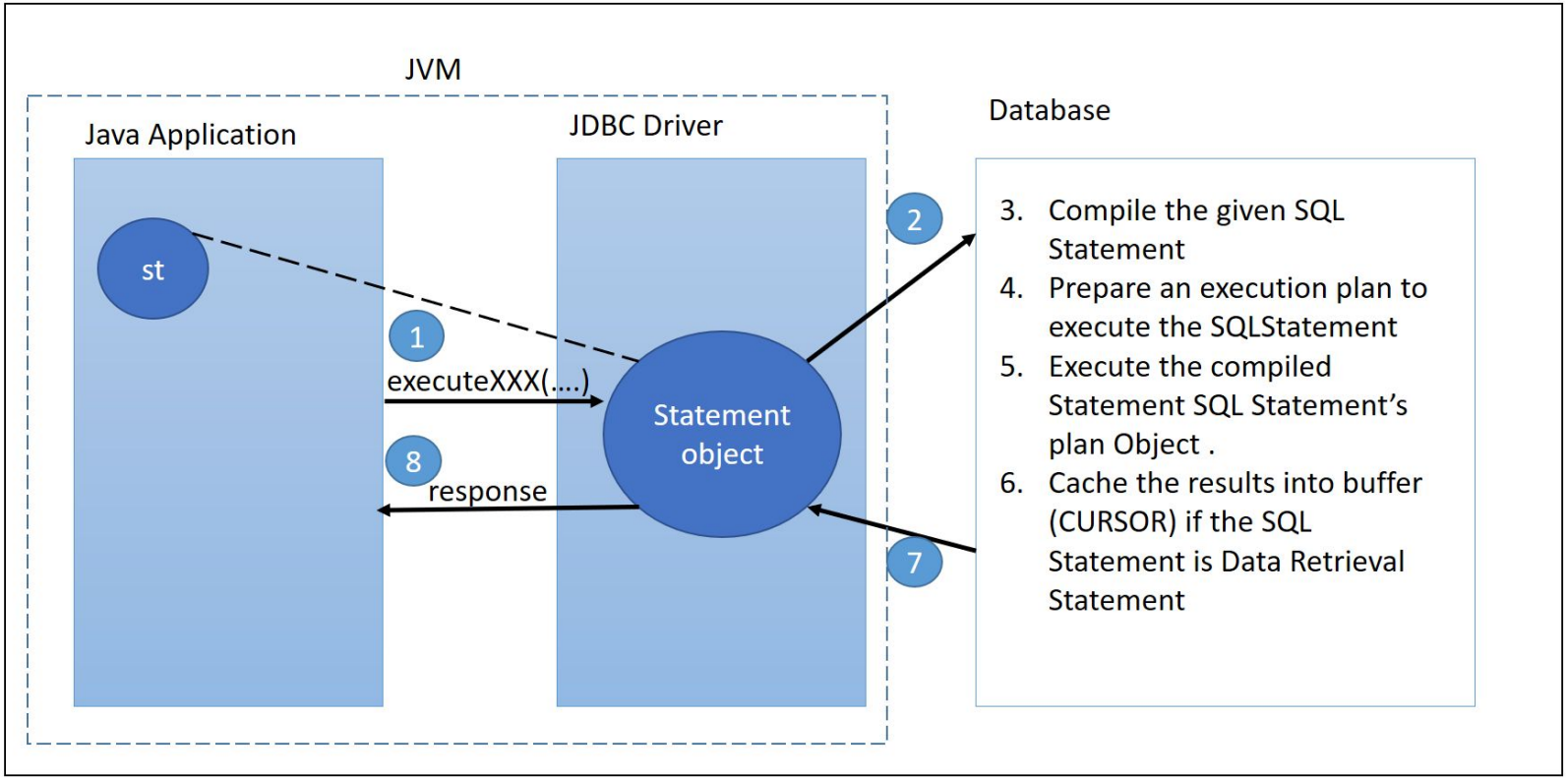
Steps Creating a Simple JDBC Application



Programming Time : 1. Check Connection with Database

Source of Image: Head First Java Page no 474

Displaying the Process Flow of the Statement Object



Programming Time : 2. Execute insert, update and delete sql query using Statement

Source of Image: Head First Java Page no 474

PreparedStatement Interface

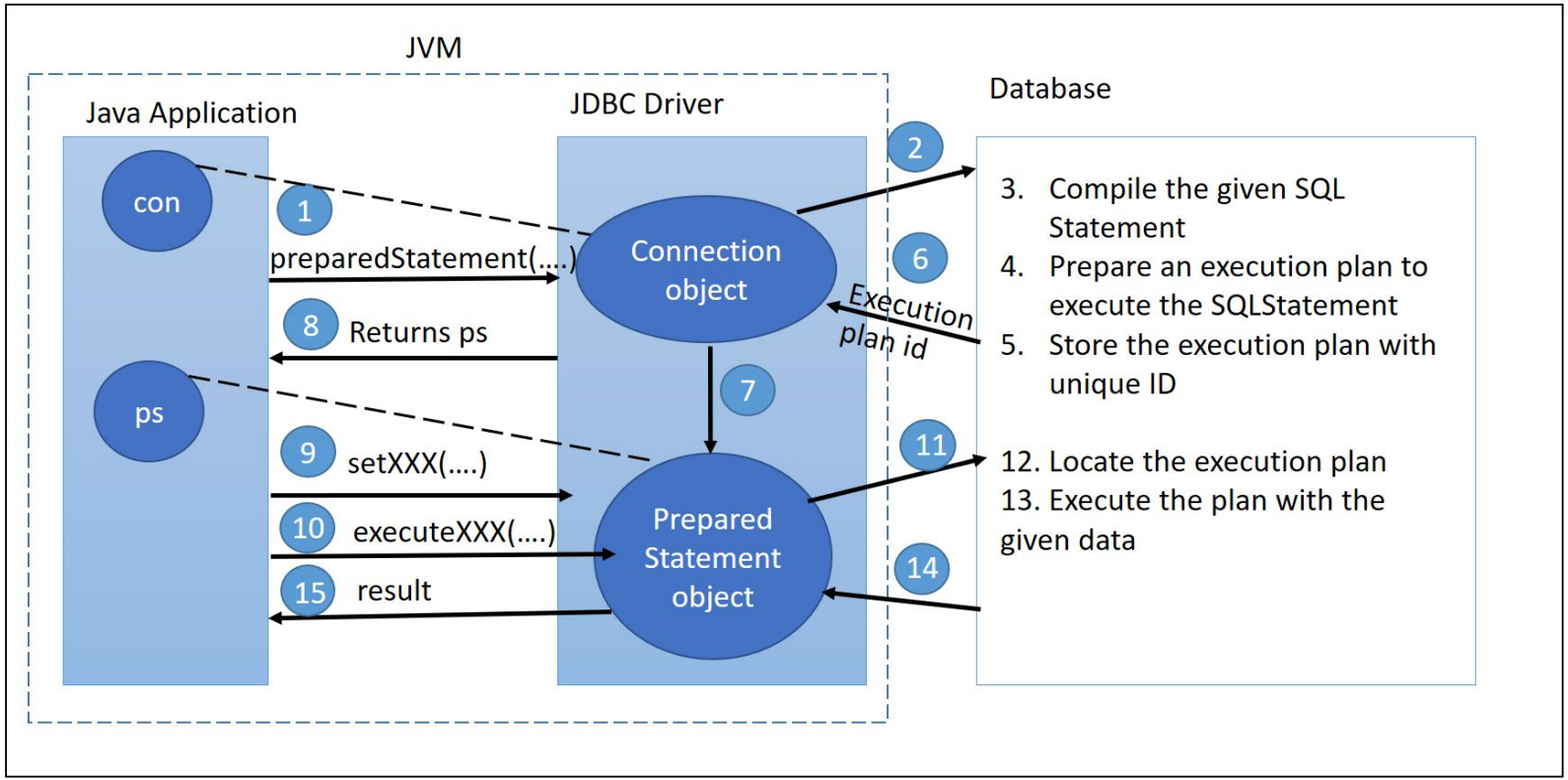
- An object that represents a precompiled SQL statement.
- A SQL statement is precompiled and stored in a PreparedStatement object.
- This object can then be used to efficiently execute this statement multiple times.

Type	Method	Description
boolean	**execute**()	Executes the SQL statement in this PreparedStatement object, which may be any kind of SQL statement.
ResultSet	**executeQuery**()	Executes the SQL query in this PreparedStatement object and returns the ResultSet object generated by the query.
int	**executeUpdate**()	Executes the SQL statement in this PreparedStatement object, which must be an SQL Data Manipulation Language (DML) statement, such as INSERT, UPDATE or DELETE; or an SQL statement that returns nothing, such as a DDL statement.

PreparedStatement Interface

Type	Method	Description
void	**setDouble**(int parameterIndex, double x)	Sets the designated parameter to the given Java double value.
void	**setFloat**(int parameterIndex, float x)	Sets the designated parameter to the given Java float value.
void	**setInt**(int parameterIndex, int x)	Sets the designated parameter to the given Java int value.
void	**setLong**(int parameterIndex, long x)	Sets the designated parameter to the given Java long value.

Displaying the Process Flow of the PreparedStatement Object



Programming Time : 3. Execute insert, update and delete sql query using PreparedStatement

Source of Image: Head First Java Page no 474

Callablestatement Interface

- The interface used to execute SQL stored procedures.
- The JDBC API provides a stored procedure SQL escape syntax that allows stored procedures to be called in a standard way for all RDBMSs.

Type	Method	Description
`void`	**`setDouble`**`(int parameterIndex, double x)`	Sets the designated parameter to the given Java `double` value.
`void`	**`setFloat`**`(int parameterIndex, float x)`	Sets the designated parameter to the given Java `float` value.
`void`	**`setInt`**`(int parameterIndex, int x)`	Sets the designated parameter to the given Java `int` value.
`void`	**`setLong`**`(int parameterIndex, long x)`	Sets the designated parameter to the given Java `long` value.

Callablestatement Interface

Type	Method	Description
double	**getDouble**(int parameterIndex)	Retrieves the value of the designated JDBC DOUBLE parameter as a double in the Java programming language.
double	**getDouble**(**String** parameterName)	Retrieves the value of a JDBC DOUBLE parameter as a double in the Java programming language.
float	**getFloat**(int parameterIndex)	Retrieves the value of the designated JDBC FLOAT parameter as a float in the Java programming language.
float	**getFloat**(**String** parameterName)	Retrieves the value of a JDBC FLOAT parameter as a float in the Java programming language.
void	**registerOutParameter**(int parameterIndex, int sqlType)	Registers the OUT parameter in ordinal position parameterIndex to the JDBC type sqlType.

Programming Time : 4. Create stored procedures

Source of Image: Head First Java Page no 474

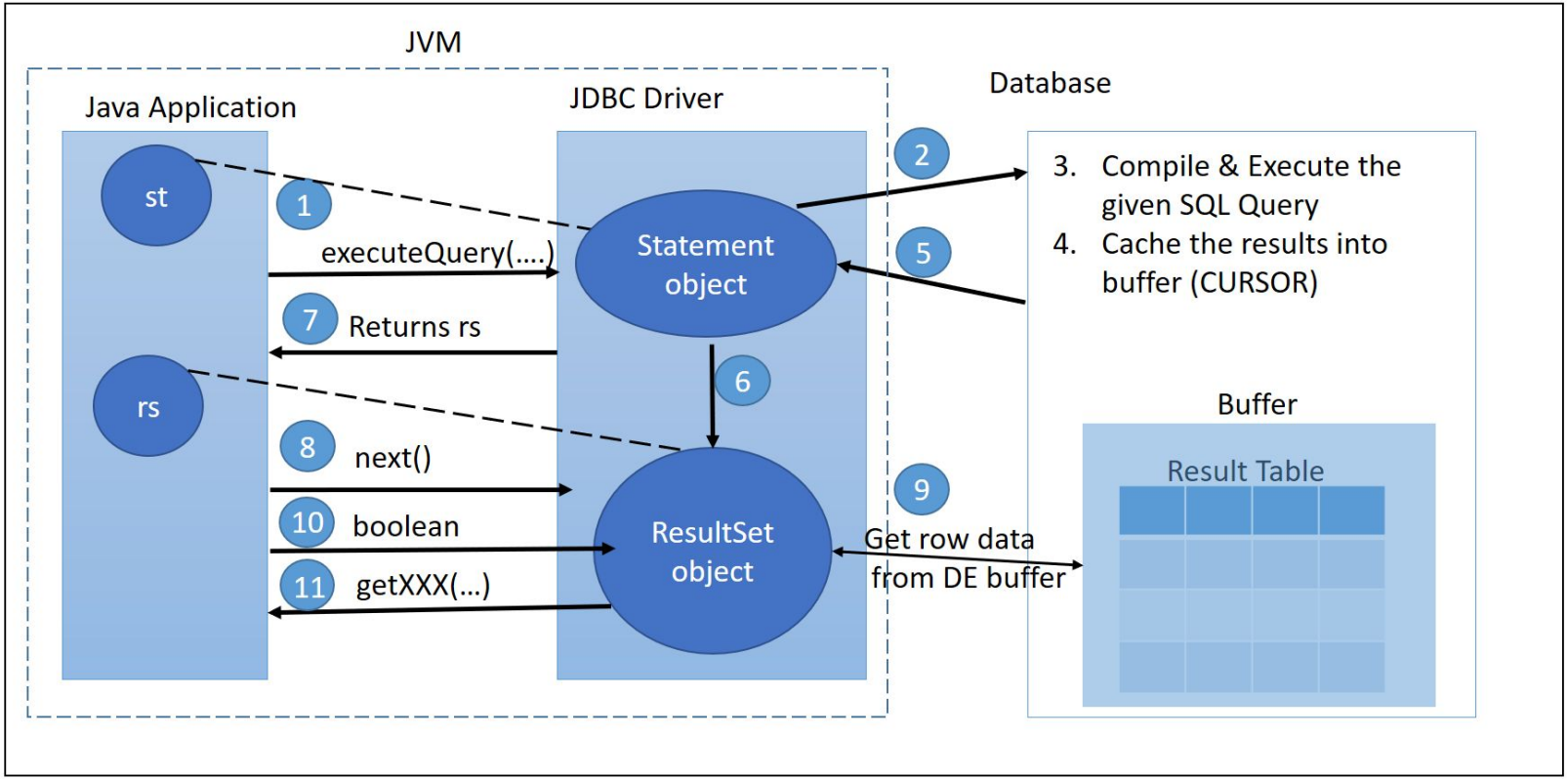
Programming Time : 5. Execute stored procedures

Source of Image: Head First Java Page no 474

ResultSet Interface

- A table of data representing a database result set, which is usually generated by executing a statement that queries the database.
- A ResultSet object maintains a cursor pointing to its current row of data.
- Initially the cursor is positioned before the first row.
- The next method moves the cursor to the next row, and because it returns false when there are no more rows in the ResultSet object, it can be used in a while loop to iterate through the result set.

ResultSet Interface



ResultSet Interface

Type	Method	Description
boolean	**absolute**(int row)	Moves the cursor to the given row number in this ResultSet object.
void	**afterLast**()	Moves the cursor to the end of this ResultSet object, just after the last row.
void	**beforeFirst**()	Moves the cursor to the front of this ResultSet object, just before the first row.
void	**cancelRowUpdates**()	Cancels the updates made to the current row in this ResultSet object.
void	**deleteRow**()	Deletes the current row from this ResultSet object and from the underlying database.

ResultSet Interface

Type	Method	Description
`int`	**`getInt`**`(int columnIndex)`	Retrieves the value of the designated column in the current row of this `ResultSet` object as an `int` in the Java programming language.
`int`	**`getInt`**`(`**`String`**` columnLabel)`	Retrieves the value of the designated column in the current row of this `ResultSet` object as an `int` in the Java programming language.
`float`	**`getFloat`**`(int columnIndex)`	Retrieves the value of the designated column in the current row of this `ResultSet` object as a `float` in the Java programming language.
`float`	**`getFloat`**`(`**`String`**` columnLabel)`	Retrieves the value of the designated column in the current row of this `ResultSet` object as a `float` in the Java programming language.
`String`	**`getString`**`(int columnIndex)`	Retrieves the value of the designated column in the current row of this `ResultSet` object as a `String` in the Java programming language.
`String`	**`getString`**`(`**`String`**` columnLabel)`	Retrieves the value of the designated column in the current row of this `ResultSet` object as a `String` in the Java programming language.

Programming Time : 6. Execute select query

Source of Image: Head First Java Page no 474

Batch Processing

- The interface used to execute SQL stored procedures.
- The JDBC API provides a stored procedure SQL escape syntax that allows stored procedures to be called in a standard way for all RDBMSs.

Type	Method	Description
`void`	**`setDouble`**`(int parameterIndex, double x)`	Sets the designated parameter to the given Java `double` value.
`void`	**`setFloat`**`(int parameterIndex, float x)`	Sets the designated parameter to the given Java `float` value.
`void`	**`setInt`**`(int parameterIndex, int x)`	Sets the designated parameter to the given Java `int` value.
`void`	**`setLong`**`(int parameterIndex, long x)`	Sets the designated parameter to the given Java `long` value.

Statement Interface

Type	Method	Description
`void`	**`addBatch`**`(`**`String`** `sql)`	Adds the given SQL command to the current list of commands for this `Statement` object.
`void`	**`clearBatch`**`()`	Empties this `Statement` object's current list of SQL commands.
`int[]`	**`executeBatch`**`()`	Submits a batch of commands to the database for execution and if all commands execute successfully, returns an array of update counts.

Programming Time : 7. Perform Batch Processing

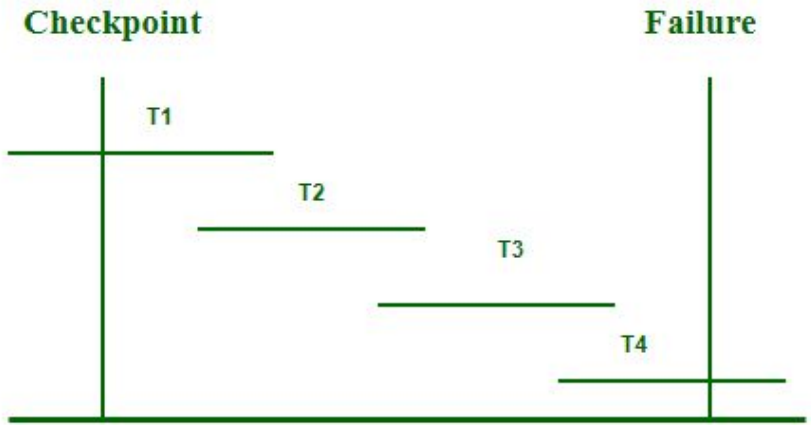
Source of Image: Head First Java Page no 474

DatabaseMetaData Interface

Type	Method	Description
`int`	**`getDatabaseMajorVersion`**`()`	Retrieves the major version number of the underlying database.
`int`	**`getDatabaseMinorVersion`**`()`	Retrieves the minor version number of the underlying database.
`String`	**`getDatabaseProductName`**`()`	Retrieves the name of this database product.
`String`	**`getDatabaseProductVersion`**`()`	Retrieves the version number of this database product.
`DatabaseMetaData`	**`getMetaData`**`()`	Retrieves a `DatabaseMetaData` object that contains metadata about the database to which this `Connection` object represents a connection.

Transaction Management

- The interface used to execute SQL stored procedures.
- The JDBC API provides a stored procedure SQL escape syntax that allows stored procedures to be called in a standard way for all RDBMSs.



Connection Interface

Type	Method	Description
Savepoint	setSavepoint()	Creates an unnamed savepoint in the current transaction and returns the new Savepoint object that represents it.
Savepoint	setSavepoint(String name)	Creates a savepoint with the given name in the current transaction and returns the new Savepoint object that represents it.
void	setAutoCommit(boolean autoCommit)	Sets this connection's auto-commit mode to the given state.
void	rollback()	Undoes all changes made in the current transaction and releases any database locks currently held by this Connection object.
void	rollback(Savepoint savepoint)	Undoes all changes made after the given Savepoint object was set.
void	releaseSavepoint(Savepoint savepoint)	Removes the specified Savepoint and subsequent Savepoint objects from the current transaction.
void	commit()	Makes all changes made since the previous commit/rollback permanent and releases any database locks currently held by this Connection object.

Programming Time : 8. Perform Transaction Management

Source of Image: Head First Java Page no 474

Add On Concepts

Source of Image: Head First Java Page no 474

DatabaseMetaData

Source of Image: Head First Java Page no 474

ResultSetMetaData

Source of Image: Head First Java Page no 474

Thank You..